🚗 Autonomous Driving Hackathon: NuScenes Mini Challenge

Welcome to the Autonomous Driving Challenge! Your task is to build a lightweight, 3-stage perception and prediction pipeline using real-world autonomous driving data.

Due to hardware and time constraints, **training deep learning models from scratch is strictly prohibited.** You must rely on clever engineering, geometry, classical physics, and small pre-trained inference models to solve these tasks.

📋 The Rules & Constraints

1. **Compute Limit:** Your entire pipeline must be able to run on a standard Kaggle or Google Colab free-tier GPU.
2. **No Deep Learning Training:** You are encouraged to *avoid* attempting any training to save time and avoid mistakes.
3. **Pre-trained Models Allowed:** You may use small, off-the-shelf pre-trained models (like YOLO) strictly for 2D inference. Minor fine tuning is allowed but is discouraged.
4. **Data:** You will use a subset of the official NuScenes v1.0-mini dataset. You are provided with 6 scenes containing full ground-truth annotations for development.
5. **Evaluation:** Final scoring will be run against a blind test set of 4 scenes.

🛠️ The 3 Tasks & Methodologies

Task 1: 2D Object Detection

The Goal: Detect vehicles in the front-facing camera images and draw 2D bounding boxes around them [xmin, ymin, xmax, ymax].

- **The Catch:** NuScenes is a 3D dataset; there are no native 2D ground truth boxes. You will need to use the provided Hackathon_Starter_Kit.ipynb to mathematically project the 3D ground truth boxes onto the 2D camera plane using the camera's intrinsic matrix. This will allow you to score your own 2D detections during development.
- **Suggested Methodology:** Run a pre-trained 2D object detector like YOLOv8 on the samples/CAM_FRONT images.

Task 2: 3D Object Detection

The Goal: Elevate your 2D detections into the 3D world. Predict the 3D center [X, Y, Z] (in meters) and the 3D size [Width, Length, Height] of each detected vehicle.

- **Suggested Methodology (The Frustum Approach):** Since you cannot train a 3D neural network, use geometry!
    1. Take the 2D bounding box you found in Task 1.
    2. Project the scene's LiDAR point cloud onto the camera image.
    3. Filter out all LiDAR points *except* the ones that land inside your 2D bounding box.
    4. You now have a 3D cluster of points belonging to that specific car. Use classical algorithms (like finding the min/max extents, or using DBSCAN from scikit-learn to remove outliers) to draw a 3D box around those points.

Task 3: Trajectory Prediction

The Goal: Given the past 2 seconds of a vehicle's movement, predict exactly where it will be over the next 6 seconds.

- **The Data:** NuScenes keyframes are captured every 0.5 seconds. You will look at the past 4 frames (2 seconds) and predict the [X, Y] coordinates for the next 12 frames (6 seconds).
- **Suggested Methodology:** Without deep learning, you must rely on physics-based kinematic models. Implement a Constant Velocity model, a Constant Acceleration model, or a basic Kalman Filter to extrapolate the future path based on the history.

📦 Required Libraries

Make sure your environment has the following installed:

- nuscenes-devkit (Crucial for reading the database)
- numpy (For matrix math and 3D projections)
- scikit-learn (For point-cloud clustering)
- matplotlib & Pillow (For visualization)
- ultralytics (If using YOLO for Task 1)
- filterpy (Optional: Highly recommended if building Kalman Filters for Task 3)

📝 Submission Format

Your final submission must be a **single file** named predictions.json. It must perfectly match the given schema. If your JSON is malformed and crashes the evaluation script, you will receive a score of zero. The example output can be found in the files you are given.

(Note: box_3d_center and box_3d_size must be in meters. trajectories must contain exactly 12 [X, Y] pairs).

🏆 Evaluation Metrics

Your predictions.json will be run against a hidden ground-truth database.

- **Task 1 (2D Detection) - F1 Score:** We calculate Intersection over Union (IoU) with a threshold of 0.5. Your score is based on the F1 metric, meaning you are rewarded for accurate boxes, but **heavily penalized** for spamming false positives or missing cars entirely. (Scale: 0 to 1.0, Higher is better).
- **Task 2 (3D Detection) - Mean Center Error:** For every correctly identified 2D car, we measure the Euclidean distance between your predicted 3D center and the actual 3D center. (Measured in meters, Lower is better).
- **Task 3 (Trajectory) - Average Displacement Error (ADE):** We calculate the average distance between your predicted path points and the actual path the car took over the 6-second horizon. (Measured in meters, Lower is better).